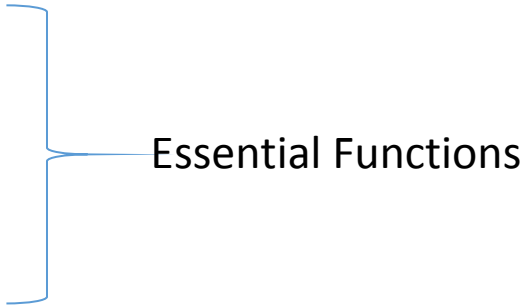
Chapter 2 Continue
+
Chapter 3

Recall: OS major functionality

1. Processor Scheduling
2. Process Management
3. Interprocess communication
4. Memory management
5. Secondary storage Management
6. I/O devices



Kernel of OS

- One of the reasons leading to modern OS development is the OS kernel.
- The kernel is a computer program at the core of a computer's operating system
- It has the complete control over everything in the system. It manages the operations of the computer and the hardware.
- Types of kernels
  - Microkernel
  - Monolithic kernel

Monolithic kernel

- the entire operating system runs as a single program in kernel mode.
- The user services and kernel services are implemented in the same address space.
- The entire operating system, including core functions
  - like memory management,
  - process management,
  - device drivers,
  - and file systems,
- All of these integrated into a single large block of code running in a single address space.

Monolithic kernel

- This design can make the system faster because all components can interact directly,

- but it can also make the system more complex and harder to maintain,
  - as a bug in one part of the kernel can potentially affect the entire system.

Microkernel

- The user services and kernel services are implemented in different address spaces.
- The user services are kept in the user address space,
- And kernel services are kept under the kernel address space.
- In a microkernel architecture, only the most essential functions are included in the kernel, such as basic communication between hardware and software, and simple process management.
- Other services like device drivers, file systems, and network protocols are run in user space as separate processes.

Microkernel

- Size of Micro-Kernel is smaller, so it is easy to use.
- The execution of Micro-Kernel is slower.
- Only the most important services are present inside the kernel and rest of the operating systems are present inside system application program.
- The communication between client process & services running in user address space is established through message that further reduces the speed of execution.

Feature	Monolithic Kernel	Microkernel
Structure	All system services (file system, drivers, memory management, etc.) run inside the kernel space.	Only the essential functions (process management, IPC, low-level hardware control) are inside the kernel; other services run in user space.
Performance	Faster — everything runs in one space without communication overhead.	Slower — because it relies on message passing (IPC) between kernel and user services.
Design	Complex and hard to modify since all parts are tightly integrated.	Modular and easier to maintain or update.
Stability	Less stable — a bug in any kernel service can crash the whole system.	More stable — a failure in one service doesn't affect the kernel.
Security	Less secure — all code runs with kernel privileges.	More secure — services run separately in user mode.
Kernel Size	Large.	Very small.
Examples	Linux, UNIX, MS-DOS.	Minix, QNX, Mach, L4.
Communication	Direct function calls inside the kernel.	Message passing (Inter-Process Communication).
Ease of Adding New Features	Difficult — changes require modifying kernel code.	Easy — new services can be added outside the kernel.

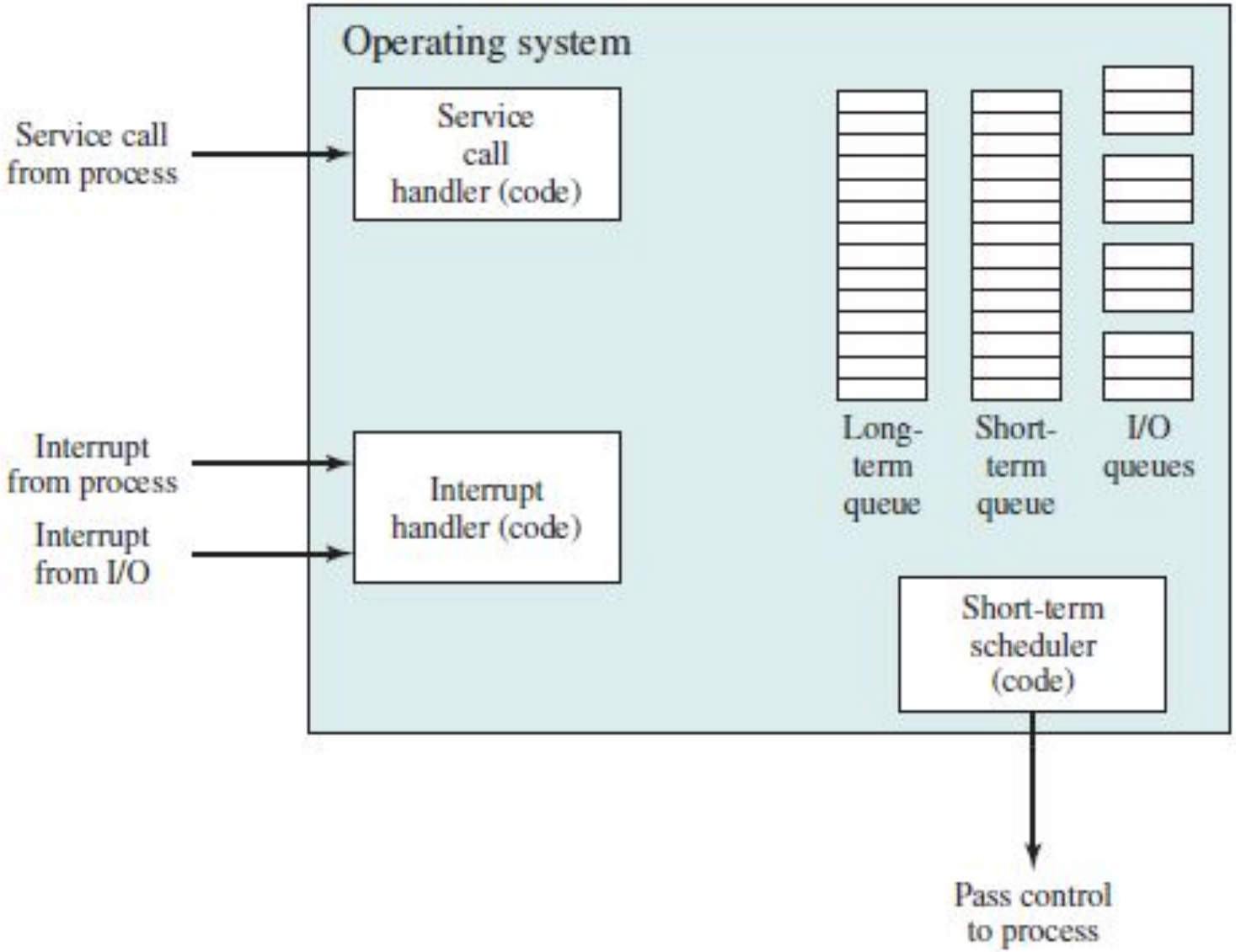
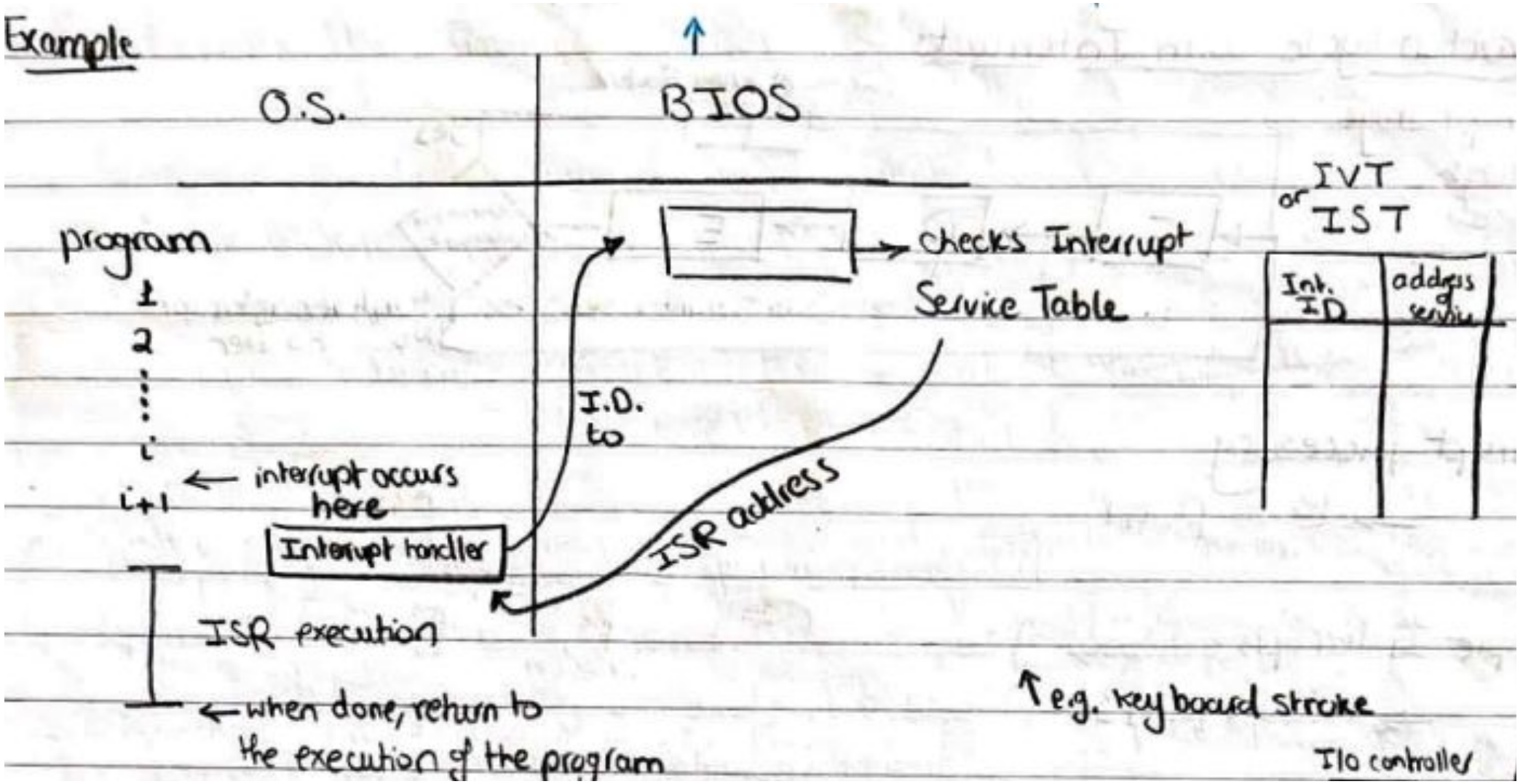


Figure 2.11 Key Elements of an Operating System for Multiprogramming

Recall: interrupts

- Software interrupts:
  - A software interrupt is a mechanism used by software to signal the processor that it needs its attention.
  - It is a way for a program to request a service from the operating system or to signal an event.
- Hardware interrupts:
  - Hardware interrupts are part of the BIOS
  - Caused by some hardware device such as a request to start an I/O, a hardware failure, or something similar.
  - Hardware interrupts were introduced as a way to avoid wasting the processor's valuable time in polling loops and waiting for external events.
  - For example, when an I/O operation is completed such as reading some data into the computer from a tape drive.

Example


O.S.
BIOS
program
1
2
i
i+1
interrupt occurs here
Interrupt handler
checks Interrupt Service Table
I.D. to
ISR address
ISR execution
when done, return to the execution of the program
IVT or IST
Int. ID
address
e.g. key board stroke
I/O controller

Multiprogramming vs Multithreading

- Multi-programming is more than one process running at a time, it increases CPU utilization by organizing jobs (code and data) so that the CPU always has one to execute.

- Multithreading is a technique such that multiple threads are created of a process for increasing the computing speed of the system.

Multiprocessing vs Multicore

- **Multicore System:**A processor that has more than one core is called a Multicore Processor while one with a single core is called Unicore Processor or Uniprocessor. Nowadays, most systems have four cores (Quad-core) or eight cores (Octa-core). These cores can individually read and execute program instructions, giving feel like a computer system has several processors but in reality, they are cores and not processors. Each core has its own cache and a common L3 level cahe.
- Multiprocessor: Two or more processors or CPUs present in same computer, sharing system bus, memory and I/O. It allows parallel execution of different processors.

Chapter 3

Process

3.1:**Processes and Process Control Blocks**

- A program in execution.
- An instance of a program running on a computer.
- The entity that can be assigned to and executed on a processor.
- A unit of activity characterized by the execution of a sequence of instructions , a current state, and an associated set of system resources.

Process Characteristics

- **Identifier:** unique distinguisher of the process
- **State**
- **Priority:** Priority level relative to other processes.
- **Program counter:** The address of the next instruction
- **Memory pointers:** Include pointers to the program code and data associated with this process.
- **Context data:** These are data that are present in registers in the processor while the process is executing.
- **I/O status information**
- **Accounting information: m**ay include the amount of processor time and clock time used, time limits…

process control block

- The information in the preceding list is stored in a data structure, typically called a **process control block** that is created and managed by the OS.
- Thus, we can say that a process consists of program code and associated data plus a process control block.

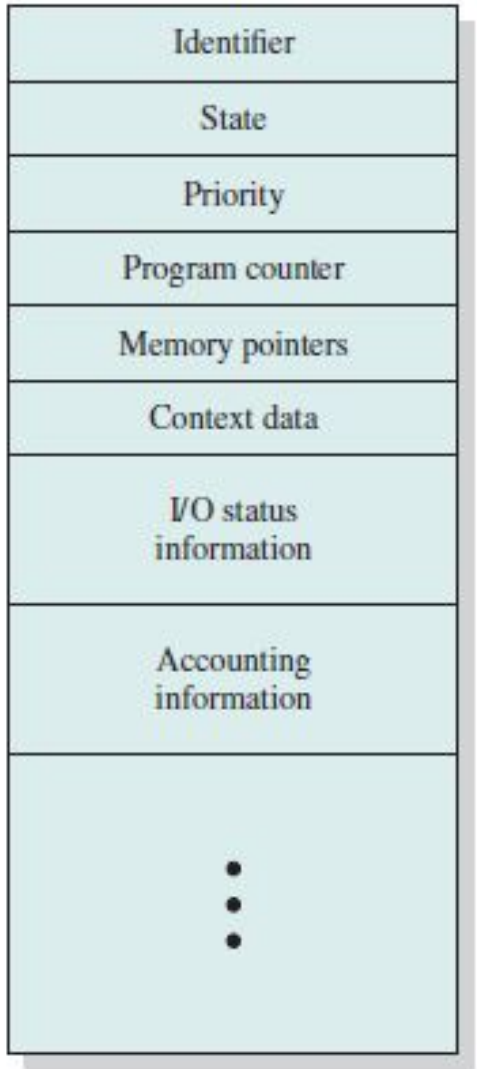


Figure 3.1 Simplified Process Control Block

A Two-State Process Model

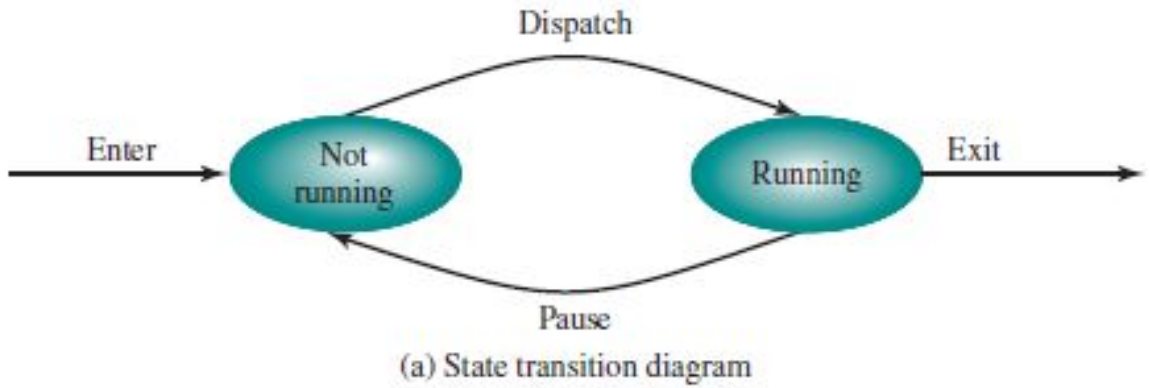


Figure 3.5 Two-State Process Model

3.2 Process States

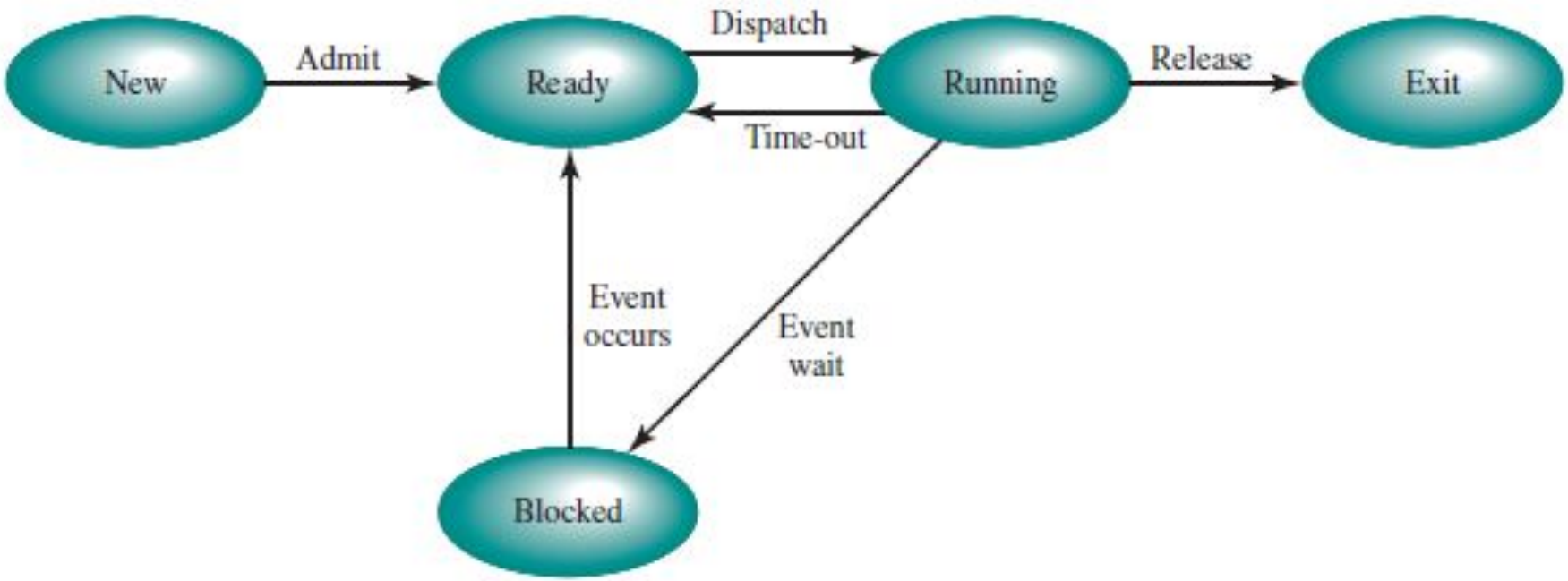


Figure 3.6 Five-State Process Model

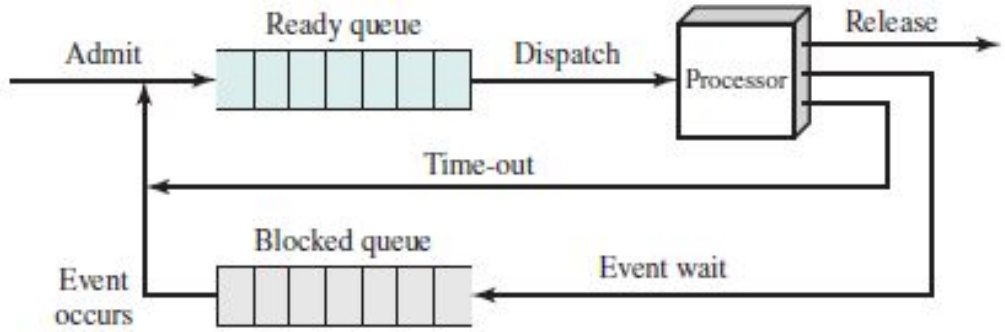
Five main process states

1. **Running:** The process that is currently being executed.
2. **Ready:** A process that is prepared to execute when given the opportunity.
3. **Blocked/Waiting:** A process that cannot execute until some event occurs, such as the completion of an I/O operation.

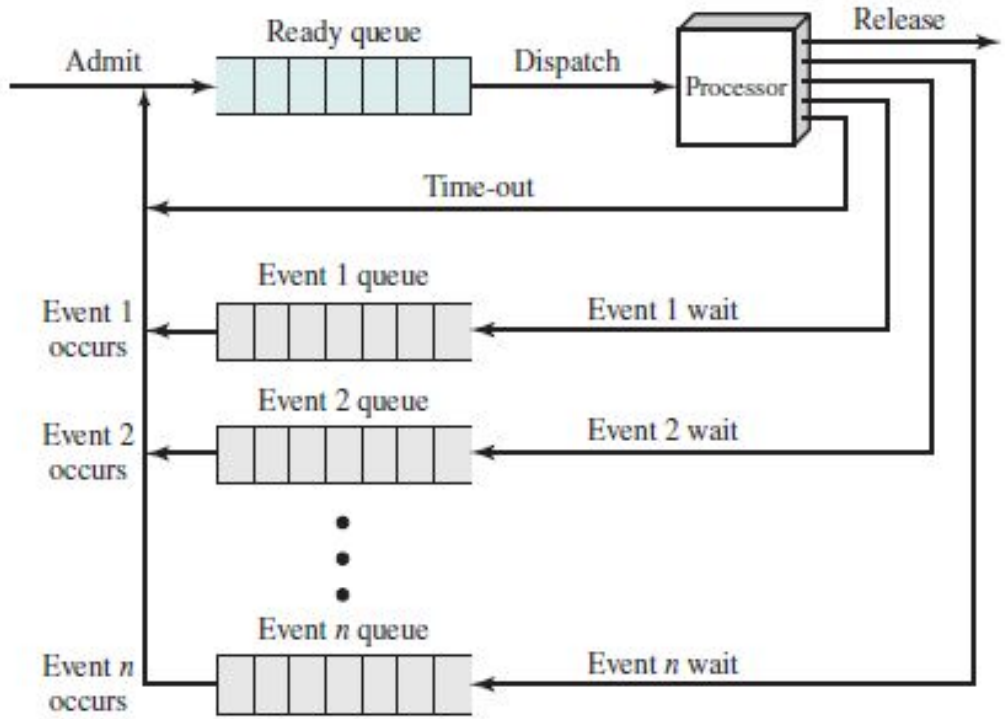
Five main process states

4. New: A process that has just been created but has not yet been admitted to the pool of executable processes by the OS. Typically, a new process has not yet been loaded into main memory, although its process control block has been created.

5. Exit: A process that has been released from the pool of executable processes by the OS, either because it halted or because it aborted for some reason.



(a) Single blocked queue



(b) Multiple blocked queues

Figure 3.8 Queueing Model for Figure 3.6

Reasons for Process Creation

Table 3.1 Reasons for Process Creation

New batch job	The OS is provided with a batch job control stream, usually on tape or disk. When the OS is prepared to take on new work, it will read the next sequence of job control commands.
Interactive log-on	A user at a terminal logs on to the system.
Created by OS to provide a service	The OS can create a process to perform a function on behalf of a user program, without the user having to wait (e.g., a process to control printing).
Spawned by existing process	For purposes of modularity or to exploit parallelism, a user program can dictate the creation of a number of processes.

Reasons for process termination

Table 3.2 Reasons for Process Termination

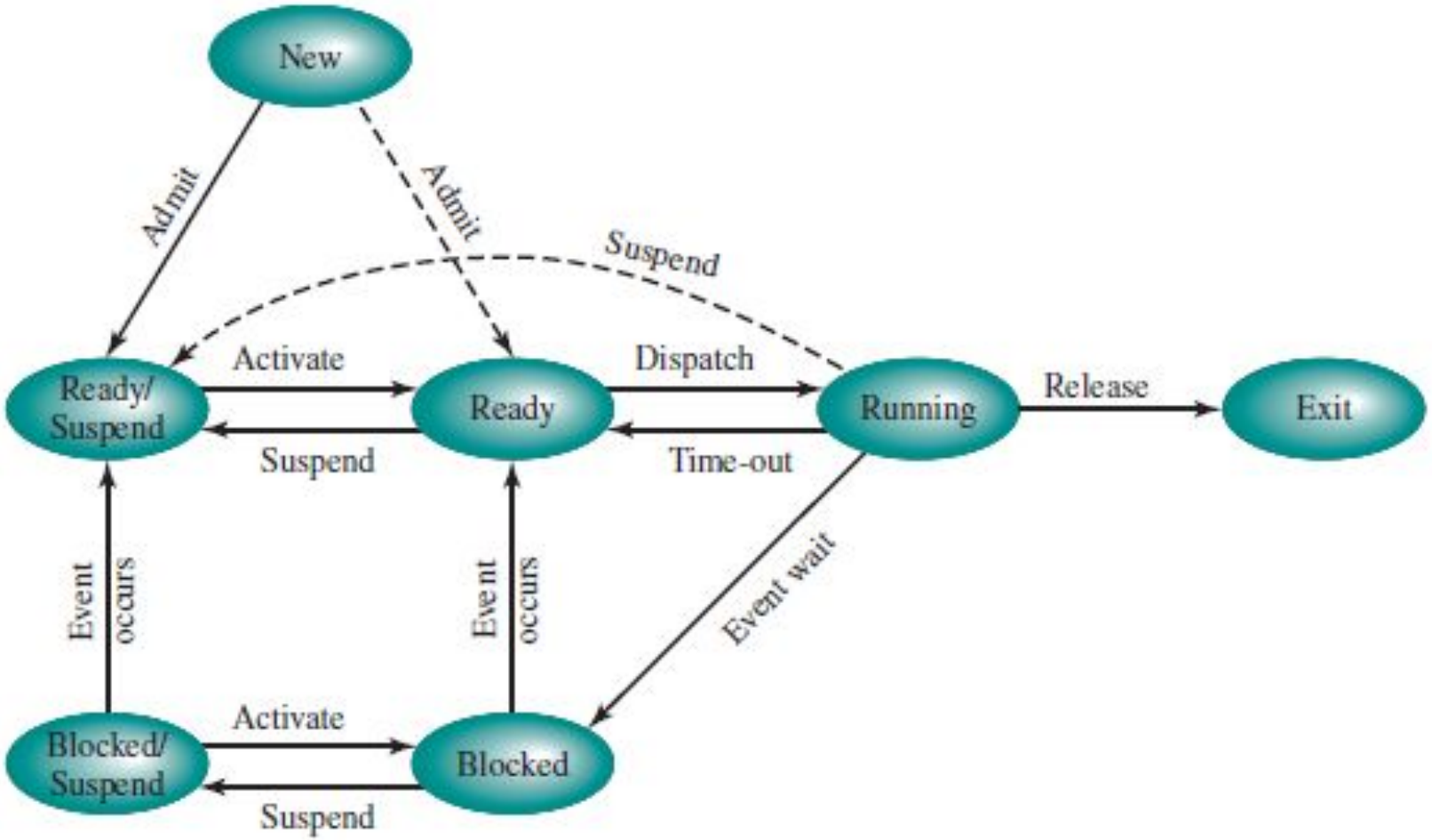
Normal completion	The process executes an OS service call to indicate that it has completed running.
Time limit exceeded	The process has run longer than the specified total time limit. There are a number of possibilities for the type of time that is measured. These include total elapsed time ("wall clock time"), amount of time spent executing, and, in the case of an interactive process, the amount of time since the user last provided any input.
Memory unavailable	The process requires more memory than the system can provide.
Bounds violation	The process tries to access a memory location that it is not allowed to access.
Protection error	The process attempts to use a resource such as a file that it is not allowed to use, or it tries to use it in an improper fashion, such as writing to a read-only file.
Arithmetic error	The process tries a prohibited computation (such as division by zero) or tries to store numbers larger than the hardware can accommodate.
Time overrun	The process has waited longer than a specified maximum for a certain event to occur.
I/O failure	An error occurs during input or output, such as inability to find a file, failure to read or write after a specified maximum number of tries (when, for example, a defective area is encountered on a tape), or invalid operation (such as reading from the line printer).
Invalid instruction	The process attempts to execute a nonexistent instruction (often a result of branching into a data area and attempting to execute the data).
Privileged instruction	The process attempts to use an instruction reserved for the operating system.
Data misuse	A piece of data is of the wrong type or is not initialized.
Operator or OS intervention	For some reason, the operator or the operating system has terminated the process (e.g., if a deadlock exists).
Parent termination	When a parent terminates, the operating system may automatically terminate all of the offspring of that parent.
Parent request	A parent process typically has the authority to terminate any of its offspring.

Suspended Processes

- Each process to be executed must be loaded fully into main memory.
- All of the processes in all of the queues must be resident in main memory.
- When none of the processes in main memory is in the Ready state, the OS swaps one of the blocked processes out on to disk into a suspend queue.
- This is a queue of existing processes that have been temporarily kicked out of main memory, or suspended.
- The OS then brings in another process from the suspend queue.

Updating states

1. **Ready:** The process is in main memory and available for execution.
2. **Blocked:** The process is in main memory and awaiting an event.
3. **Blocked/Suspend:** The process is in secondary memory and awaiting an event.
4. **Ready/Suspend:** The process is in secondary memory but is available for execution as soon as it is loaded into main memory.



(b) With two Suspend states

Figure 3.9 Process State Transition Diagram with Suspend States

Reasons for suspension

Table 3.3 Reasons for Process Suspension

Swapping	The OS needs to release sufficient main memory to bring in a process that is ready to execute.
Other OS reason	The OS may suspend a background or utility process or a process that is suspected of causing a problem.
Interactive user request	A user may wish to suspend execution of a program for purposes of debugging or in connection with the use of a resource.
Timing	A process may be executed periodically (e.g., an accounting or system monitoring process) and may be suspended while waiting for the next time interval.
Parent process request	A parent process may wish to suspend execution of a descendent to examine or modify the suspended process, or to coordinate the activity of various descendants.

3.3 Process Description

- What information does the OS need to control processes and manage resources for them?

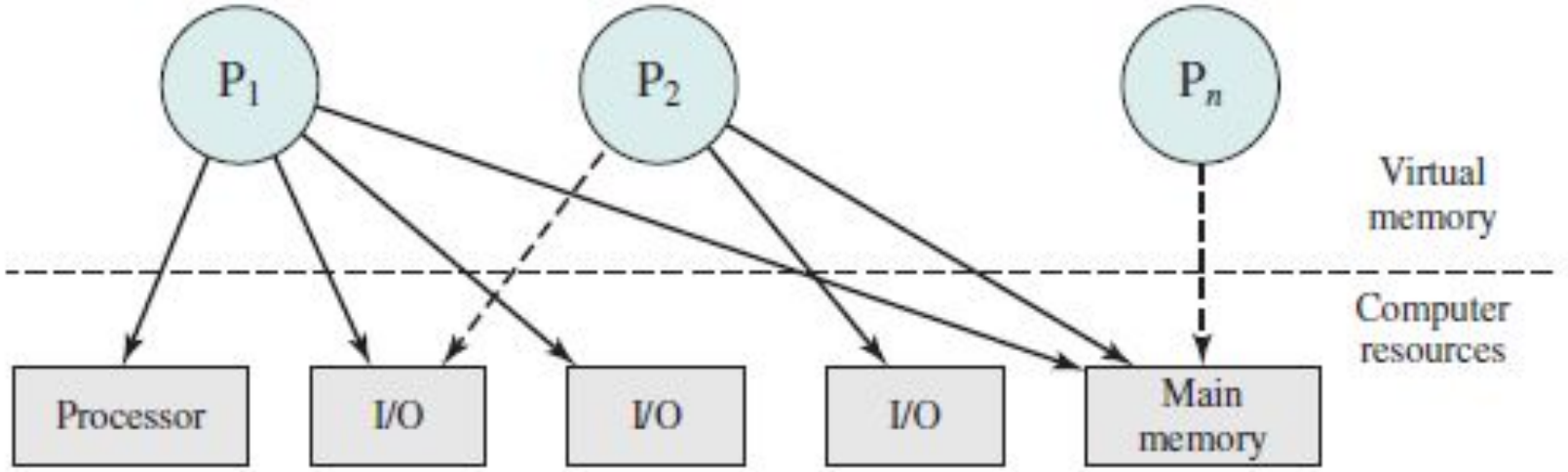


Figure 3.10 Processes and Resources (resource allocation at one snapshot in time)

Operating System Control Structures

- The OS constructs and maintains tables of information about each entity that it is managing.
- Four different types of tables maintained by the OS:
  - memory,
  - I/O,
  - file,
  - and process.

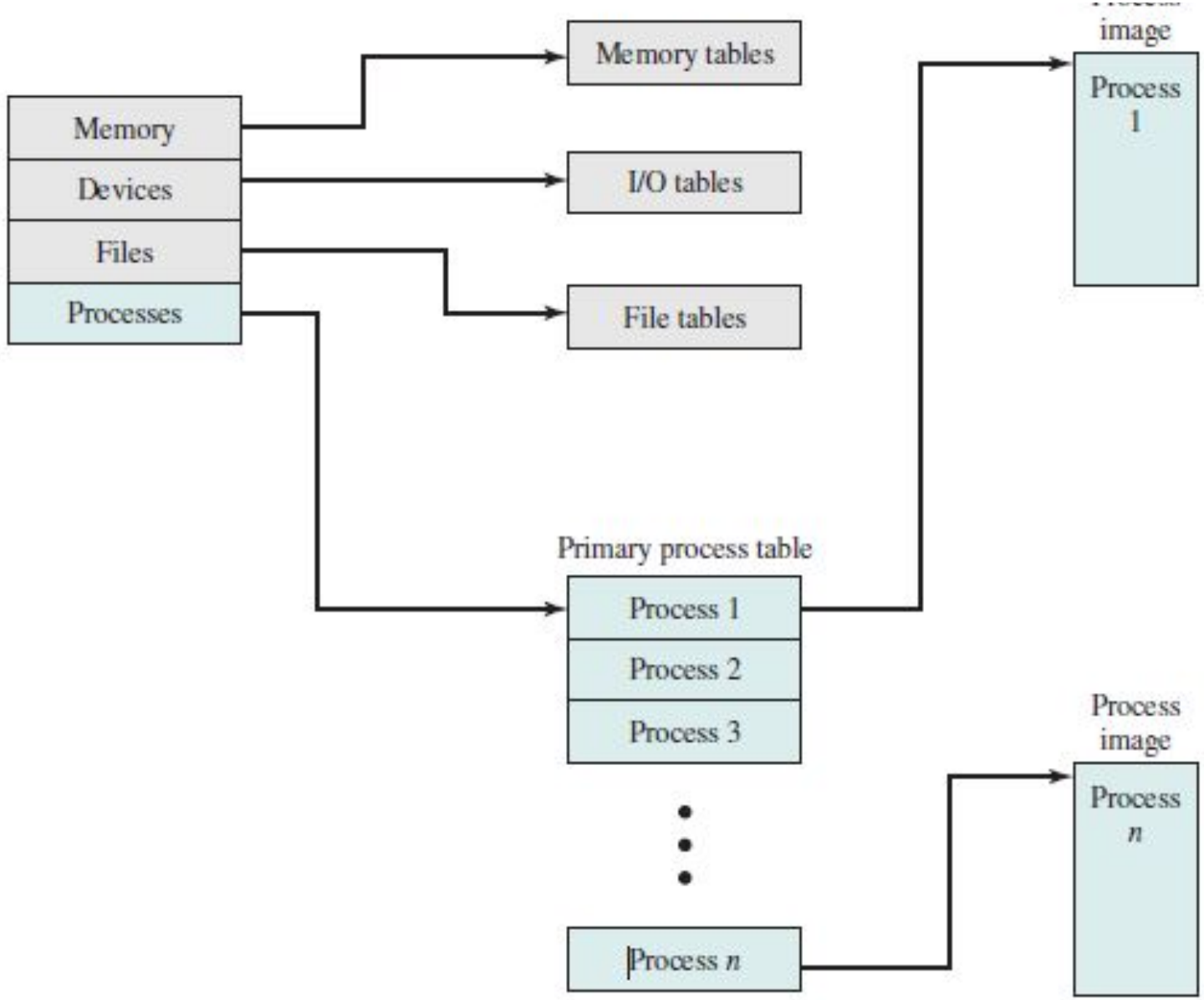


Figure 3.11 General Structure of Operating System Control Tables